04 — Advanced SQL

This module covers the most powerful SQL features used by senior engineers, data engineers, and those preparing for technical interviews at top-tier companies.

Contents

File	Topic	Difficulty
01_window_functions_intro.md	OVER, PARTITION BY, ORDER BY, frame clauses	Advanced
02_ranking_functions.md	ROW_NUMBER, RANK, DENSE_RANK, NTILE	Advanced
03_analytical_functions.md	LAG, LEAD, FIRST_VALUE, LAST_VALUE, NTH_VALUE	Advanced
04_ctes.md	Simple CTEs, multiple CTEs, CTE with DML	Advanced
05_recursive_queries.md	Recursive CTEs, trees, BOM, graphs	Expert
06_advanced_aggregation.md	Statistical functions, hypothetical aggregates	Expert
07_lateral_joins.md	LATERAL joins, top-N per group	Advanced
08_interview_window_tasks.md	20+ company-style interview problems	Expert

---

Learning Path

```
01_window_functions_intro      ← START: OVER, PARTITION BY, frame clauses
           │
           ▼
02_ranking_functions           ← ROW_NUMBER, RANK, DENSE_RANK, NTILE
           │
           ▼
03_analytical_functions        ← LAG, LEAD, FIRST_VALUE, LAST_VALUE
           │
           ├──── 04_ctes        ← WITH clause, recursive CTEs
           │         └──────────── 05_recursive_queries ← trees, graphs, BOM
           │
           ├──── 06_advanced_aggregation ← PERCENTILE, STDDEV, JSON_AGG
           │
           ├──── 07_lateral_joins ← LATERAL for correlated FROM queries
           │
           └──── 08_interview_window_tasks ← practice everything together
```

Prerequisites

- Completed `03_Intermediate_SQL/` module
- Solid understanding of GROUP BY, JOINs, subqueries
- Familiarity with aggregate functions

What You Will Be Able to Do

After completing this module:

1. Use all window functions confidently: ranking, navigation, aggregation
2. Write CTEs for readable, maintainable complex queries
3. Traverse hierarchical data (org charts, BOM, file trees) with recursive CTEs
4. Compute percentiles, medians, and statistical measures
5. Use LATERAL joins for correlated subqueries in FROM
6. Solve 20+ company-level interview problems independently

Key Patterns Reference

Running Total

```
SUM(amount) OVER (PARTITION BY region ORDER BY date ROWS BETWEEN UNBOUNDED PRECEDING
AND CURRENT ROW)
```

Top-N per Group

```
SELECT * FROM (SELECT *, ROW_NUMBER() OVER (PARTITION BY grp ORDER BY val DESC) AS
rn FROM t) x WHERE rn <= N;
```

Period-over-Period Change

```
amount - LAG(amount) OVER (PARTITION BY group ORDER BY date)
```

Gap and Island Detection

```
date - ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY date)::INT AS grp
```

Median

```
PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary)
```

Recursive Hierarchy

```
WITH RECURSIVE tree AS (
    SELECT * FROM tbl WHERE parent_id IS NULL
    UNION ALL
    SELECT t.* FROM tbl t JOIN tree ON t.parent_id = tree.id
)
SELECT * FROM tree;
```

Interview Topics Covered

- Window function vs GROUP BY aggregation
- Frame clause ROWS vs RANGE semantics
- LAST_VALUE default frame gotcha
- ROW_NUMBER vs RANK vs DENSE_RANK tie handling
- CTE materialization in PostgreSQL 12+
- Recursive CTE termination and cycle detection
- LATERAL vs correlated subquery vs window function
- Gap-and-island detection technique
- Year-over-year / month-over-month analysis patterns
- Deduplication with ROW_NUMBER

Next Module

Proceed to `05_PostgreSQL_Core/` for:

- Data types and type casting
- PostgreSQL-specific functions
- JSON and JSONB operations
- Arrays and composite types